

Reviewer Pack — Minimal Tropical Motivic Rank and Communication Complexity R...

Entry 917619ef757d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 05:44:42 UTC

Minimal Tropical Motivic Rank and Communication Complexity Rank Correlation

Entry ID: 917619ef757d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 05:44:42 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -vertex graph G with girth at least 5, the minimal tropical motivic rank ($\text{mtr}(G)$) of its associated Tseitin formula φ_G is linearly correlated with its communication complexity rank ($\text{CCR}(\varphi_G)$), such that $\text{mtr}(G) = \Theta(\text{CCR}(\varphi_G))$.

Rationale (proposer's reasoning):

Tropical geometry has shown promise in understanding circuit complexity, and motivic rank has been a key invariant. Communication complexity rank provides a measure of the difficulty of distributing information between two parties. This conjecture suggests that tropical geometric properties could be related to communication complexity, offering a potential bridge for hardness amplification.

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0ed74320e711cb8a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an n -vertex graph G with girth ≥ 5 , if the ratio of the minimal tropical motivic rank ($\text{mtr}(G)$) to its communication complexity rank ($\text{CCR}(\varphi_G)$) is bounded by a constant factor k for all seeds s , where $|k - \text{mtr}(G) / \text{CCR}(\varphi_G)| \leq \varepsilon$ across at least 95% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal tropical motivic rank AND Communication Complexity rank AND graph girth-Tseitin formula AND minimal tropical motivic rank AND communication complexity rank-girth at least 5 AND Tseitin formula AND linear correlation minimal tropical motivic rank communication complexity rank

Top relevant hits considered: - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/1605.01539v2>] Rank and select: Another lesson learned - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/2601.15543v5>] Height moduli of elliptic surfaces: Motivic height zeta rationality and Kudla-Millson modularity of Mordell-Weil rank ju - [<http://arxiv.org/abs/1110.4938v2>] A geometric approach to finite rank unitary perturbations - [<http://arxiv.org/abs/2001.05402v2>] 1-planar graphs with minimum degree at least 3 have bounded girth - [<http://arxiv.org/abs/2002.05617v3>] Waring rank of binary forms, harmonic cross-ratio and golden ratio

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations, product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        while True:
            edges = set()
            for u in range(n):
                for v in range(u + 1, n):
                    if random.random() < 0.5 and len(edges & {(u, w), (w, u) for w in range(n)}) < 2:
                        edges.add((u, v))
            G = {u: set(v for u, v in edges if u == u) for u in range(n)}
            if all(len(G[u]) >= 2 for u in range(n)) and max(len(path) for path in find_all_paths(G, n)):
                return G

    def find_all_paths(G, n):
        paths = []
        for start in range(n):
```

```

visited = [False] * n
visited[start] = True
stack = [(start, [start])]
while stack:
    u, path = stack.pop()
    if len(path) >= 5:
        paths.append(path)
    for v in G[u]:
        if not visited[v]:
            visited[v] = True
            stack.append((v, path + [v]))
return paths

def tseitin_formula(G):
    n = len(G)
    literals = {u: f'x{u}' for u in range(n)}
    neg_literals = {u: f'-x{u}' for u in range(n)}
    clauses = []

    # Clause for each edge
    for u, v in G:
        clauses.append([literals[u], neg_literals[v]])
        clauses.append([neg_literals[u], literals[v]])
        clauses.append([neg_literals[u], neg_literals[v]])

    # Clause for each vertex
    for u in range(n):
        clauses.append([literals[u]] + [neg_literals[v] for v in G[u]])

    return clauses

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(i+1, n):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(n + 1):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def minimal_tropical_motivic_rank(cnf):
    n = len(cnf)
    matrix = [[0] * (n + 1) for _ in range(n)]
    for clause in cnf:
        for literal in clause:
            if literal[0] == '-':
                u = int(literal[1:]) - 1
                matrix[u][n] += 1
            else:
                u = int(literal) - 1
                matrix[n][u] += 1

```

```

rank = sum(1 for row in gaussian_elimination(matrix) if any(row))
return rank

def communication_complexity_rank(cnf):
    n = len(cnf)
    matrix = [[0] * (n + 1) for _ in range(n)]
    for clause in cnf:
        for literal in clause:
            if literal[0] == '-':
                u = int(literal[1:]) - 1
                matrix[u][n] += 1
            else:
                u = int(literal) - 1
                matrix[n][u] += 1
    rank = sum(1 for row in gaussian_elimination(matrix) if any(row))
    return rank

n = random.choice([5, 10, 15, 20, 30, 40])
G = generate_random_graph(n)
cnf = tseitin_formula(G)

mtr = minimal_tropical_motivic_rank(cnf)
ccr = communication_complexity_rank(cnf)

if ccr == 0:
    return {
        "metric_name": "mtr_to_ccr_ratio",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "CCR is zero, division by zero"
    }

ratio = Fraction(mtr, ccr)
return {
    "metric_name": "mtr_to_ccr_ratio",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": True if abs(ratio - mtr / ccr) <= 0.01 else False,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **trial_result}}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)

```

```

std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
support_fraction = 1.0
print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(result["conjecture_holds"] is False for result in results):
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"mtr_to_ccr_ratio does not hold\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no seeds supported the conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c09ae629.py", line 129, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c09ae629.py", line 95, in run_trial
    mtr = minimal_tropical_motivic_rank(cnf)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c09ae629.py", line 68, in minimal_tropical_motivic_rank
    rank = sum(1 for row in matrix if any(row))
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'NoneType' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's validity. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21119
2	preregistration	ollama_remote	glm4:latest	0	0	9332
3	novelty	ollama_remote	glm4:latest	0	0	8517
4	novelty	ollama_remote	glm4:latest	0	0	9901
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45214

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17522
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17466
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16453
9	judge	ollama_remote	glm4:latest	0	0	14936

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160461 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/917619ef757d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/917619ef757d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/917619ef757d.tar.gz (if generated)